

Chapter 2

Chip Basics: Time, Area, Power, Reliability and Configurability

2.1 Introduction

The tradeoff between cost and performance is fundamental to any system design. Different designs result either from the selection of different points on the cost–performance continuum, or from differing assumptions about the nature of cost or performance.

The driving force in design innovation is the rapid advance in technology. The Semiconductor Industry Association (SIA) regularly makes projections, called the SIA roadmap, of technology advances which become the basis and assumptions for new chip designs. While the projections change, the advance has been and is expected to continue to be formidable. Table 2.1 is a summary of the roadmap [1] projections for the highest performing microprocessors introduced in a particular year. This projection is only for state-of-the-art processors, and not the vast number of other processors produced in that year. With the advances in lithography, the transistors are getting smaller. The minimum width of the transistor gates is defined by the process technology. Table 2.1 refers to process technology generations in terms of nanometers (nm); previously, generations are referred to in terms of microns; with earlier generations being 0.13, 0.18, 0.25 and 0.35 microns.

<i>Year</i>	2010	2013	2016
<i>Technology generation(nm)</i>	45	32	22
<i>Wafer size, diameter(cm)</i>	30	45	45
<i>Defect density(per cm²)</i>	0.14	0.14	0.14
<i>μP die size (cm²)</i>	3.1	3.1	3.1
<i>Chip Frequency (GHz)</i>	15	23?	36?
<i>Million Transistors per cm²</i>	714	1400	2800
<i>Max Power(W) High Performance</i>	198	198	198

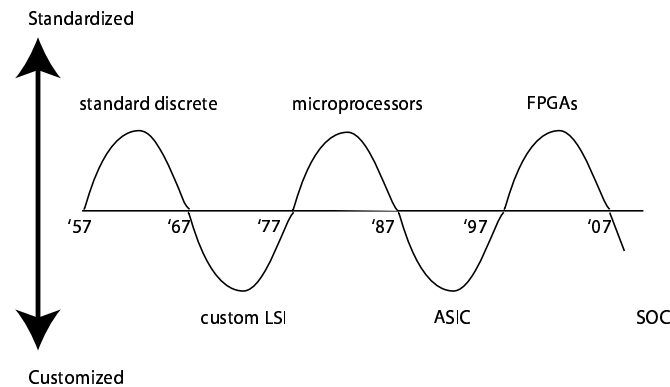
Table 2.1 Technology roadmap projections.

Five big issues in SOC design

Four of the issues are obvious. *Die area* (manufacturing cost) and *performance* (heavily influenced by cycle time) are important basic SOC design consideration. *Power consumption* has also come to the fore as a design limitation. As technology shrinks feature sizes, *reliability* will dominate as a design parameter.

The fifth issue, *Configurability*, is less obvious as an immediate design consideration. However, as we see in Chapter 1, in SOC design the non-recurring design costs dominate the total project cost. Making a design flexible through reconfigurability is an important issue to broaden the market – and reduce the per part cost – for SOC design.

Configurability enables programmability in the field, and can be seen to provide features that are “standardized in manufacturing while customized in application.” Indeed the cyclical nature of the integrated circuit industry between standardization and customization has been observed by Makimoto [2], and is known as Makimoto’s Wave: see below.



With increases in chip frequency and especially in transistor density, the designer must be able to find the best set of tradeoffs in an environment of rapidly changing technology. Already the chip frequency projections have been called into question because of the resulting power requirements.

In making basic design tradeoffs, we have five different considerations. The first is *time*, which includes partitioning instructions into events or cycles, basic pipelining mechanisms used in speeding up instruction execution, and cycle time as a parameter for optimizing program execution. Second, we discuss *area*. The cost or area occupied by a particular feature is another important aspect of the architectural tradeoff. Third, *power consumption* affects both performance and implementation. Instruction sets that require more implementation area are less valuable than instruction sets that use less—unless, of course, they can provide commensurately better performance. Long-term *cost-performance ratio* is the basis for most design decisions. Fourth, *reliability*, comes into play to cope with deep-submicron effects. Fifth, *configurability*, provides an additional opportunity for designers to trade-off recurring and non-recurring design costs.

In terms of complexity theory, various trade-offs are possible. For instance at a fixed

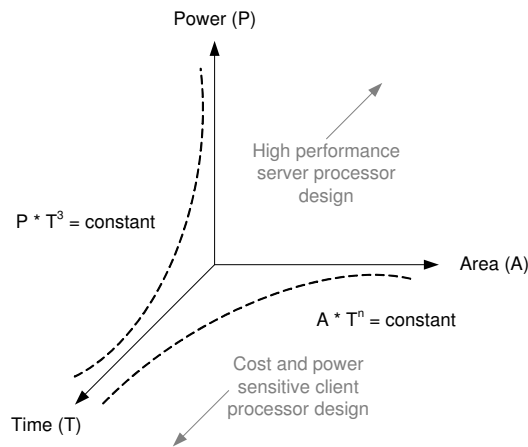


Figure 2.1 Processor design tradeoffs.

feature size, area can be traded off for performance (expressed in term of execution time, T). VLSI (Very Large Scale Integration) complexity theorists have shown that an $A \times T^n$ bound exists for processor designs, where n usually falls between 1 and 2. It is also possible to trade off time T for power P with a $P \times T^3$ bound. Figure 2.1 shows the possible trade-off involving area, time, and power in a processor design. Embedded and high-end processors operate in different design regions of this three-dimensional space. The power and area axes are typically optimized for embedded processors, whereas the time axis is typically for high-end processors.

This chapter deals with design issues in making these tradeoffs. It begins with the *processor cycle* – specifically, how much delay is incurred in a cycle and how instruction execution is partitioned into cycles. Following this, we look at the cycle time of pipelined processors. For such processors, we can approximately determine the optimum cycle time. We introduce a cost (area) model to assist in making various tradeoffs. This model is restricted to on-chip or processor-type tradeoffs, but it illustrates a type of model that the designer must use in all processor designs. We then look at reliability and reconfiguration. Finally we discuss and illustrate the impact of the five design issues on requirements and specifications.

2.2 Cycle Time

The notion of time receives considerable attention from processor designers. It is the basic measure of performance; but breaking actions into cycles and reducing both cycle count and cycle times are important but inexact sciences.

The way actions are partitioned into cycles is important. A common problem is having unanticipated “extra” cycles required by a basic action such as a cache miss. Overall, there is only a limited theoretical basis for cycle selection and the resultant partitioning of instruction execution into cycles. Much design is done on a pragmatic basis.

In this section, we look at some techniques for instruction partitioning, that is, techniques for breaking up the instruction execution time into manageable and fixed time

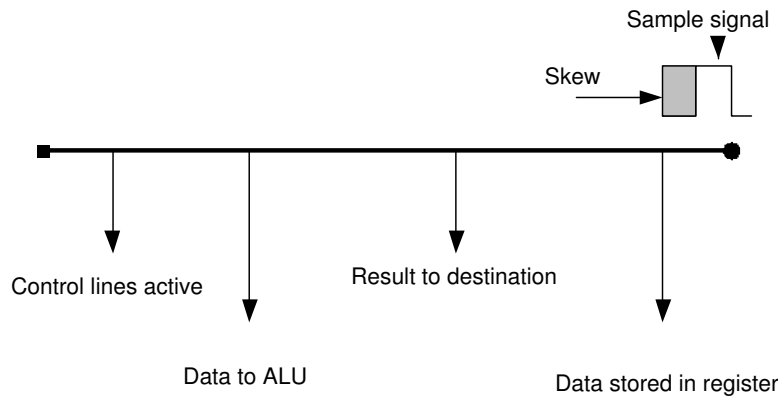


Figure 2.2 Possible sequence of actions within a cycle.

cycles. In a pipelined processor, data flows through stages much as items flow on an assembly line. At the end of each stage, a result is passed on to a subsequent stage and new data enter. Within limits, the shorter the cycle time, the more productive the pipeline. The partitioning process has its own overhead, however, and very short cycle times become dominated by this overhead. Simple cycle time models can optimize the number of pipeline stages.

2.2.1 Defining a Cycle

A cycle (of the clock) is the basic time unit for processing information. In a synchronous system the clock rate is a fixed value and the cycle time is determined by finding the maximum time to accomplish a frequent operation in the machine; such as an add or a register data transfer. This time must be sufficient for data to be stored into a specified destination register (Figure 2.2). Less frequent operations that require more time to complete require multiple cycles.

A cycle begins when the instruction decoder (based on the current instruction OP code) specifies the values for the registers in the system. These control values connect the output of a specified register to another register or an adder or similar object. This allows data from source registers to propagate through designated combinatorial logic into the destination register. Finally, after a suitable setup time, all registers are sampled by an edge or pulse produced by the clocking system.

In a synchronous system, the cycle time is determined by the sum of the worst-case time for each step or action within the cycle. However, the clock itself may not arrive at the anticipated time (due to propagation or loading effects). We call the maximum deviation from the expected time of clock arrival the (uncontrolled) *clock skew*.

In an asynchronous system, the cycle time is simply determined by the completion of an event or operation. A *completion signal* is generated, which then allows the next operation to begin. Asynchronous clocking is not generally used within pipelined processors because of the completion signal overhead and pipeline timing constraints.

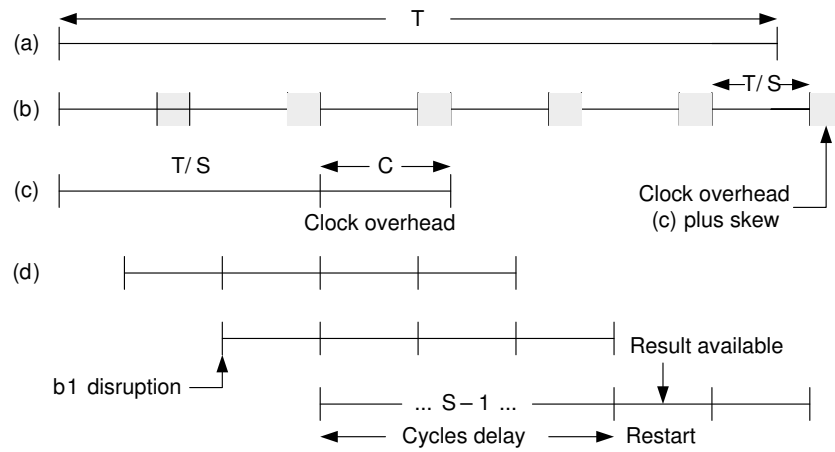


Figure 2.3 Optimal pipelining. (a) Unlocked instruction execution time, T . (b) T is partitioned into S segments. Each segment requires C clocking overhead. (c) Clocking overhead and its effect on cycle time T/S . (d) Effect of a pipeline disruption (or a stall in the pipeline).

2.2.2 Optimum Pipeline

A basic optimization for the pipeline processor designer is the partitioning of the pipeline into concurrently operating segments. A greater number of segments allow a higher maximum speedup. However, each new segment carries clocking overhead with it, which can adversely affect performance.

If we ignore the problem of fitting actions into an integer number of cycles, we can derive an optimal cycle time, Δt , and hence the level of segmentation for a simple pipelined processor.

Assume that the total time to execute an instruction without pipeline segments is T nanoseconds (Figure 2.3a). The problem is to find the optimum number of segments S of T to allow clocking and pipelining. The ideal delay through a segment is $T/S = T_{seg}$. Associated with each segment is partitioning overhead. This clock overhead time C (in ns), includes clock skew and any register requirements for data setup and hold.

Now, the actual cycle time (Figure 2.3c) of the pipelined processor is the ideal cycle time T/S plus the overhead:

$$\Delta t = \frac{T}{S} + C.$$

In our idealized pipelined processor, if there are no code delays, it processes instructions at the rate of one per cycle; but delays can occur (primarily due to incorrectly guessed or unexpected branches). Suppose these interruptions occur with frequency b and have the effect of invalidating the $S - 1$ instructions prepared to enter, or already in, the pipeline (representing a “worst-case” disruption, Figure 2.3d). There are many different types of pipeline interruption, each with a different effect, but this simple model illustrates the effect of disruptions on performance.

Considering pipeline interruption, the performance of the processor is:

$$\text{Performance} = \frac{1}{1 + (S - 1)b} \text{instructions per cycle.}$$

The throughput (G) can be defined as:

$$\begin{aligned} G &= \frac{\text{performance}}{\Delta t} \text{ instructions/ns} \\ &= \left(\frac{1}{1 + (S - 1)b} \right) \times \left(\frac{1}{T/S + C} \right). \end{aligned}$$

If we find

$$\frac{dG}{dS} = 0,$$

we can find S_{opt} , the optimum number of pipeline segments:

$$S_{\text{opt}} = \sqrt{\frac{(1 - b)T}{bC}}.$$

Once an initial S has been determined, the total instruction execution latency (T_{instr}) is:

$$T_{\text{instr}} = T + S \times (\text{clocking overhead}) = T + SC, \text{ or } S(T_{\text{seg}} + C) = S\Delta t.$$

Finally, we compute the throughput performance G in (million) instructions per second.

Suppose $T = 12.0$ ns and $b = 0.2$, $C = 0.5$ ns. Then $S_{\text{opt}} \doteq 10$ stages.

This S_{opt} as determined is simplistic—functional units cannot be arbitrarily divided, integer cycle boundaries must be observed, etc. Still, determining S_{opt} can serve as a design starting point or as an important check on an otherwise empirically optimized design.

The preceding discussion considers a number of pipeline segments S on the basis of performance. Each time a new pipeline segment is introduced, additional cost is added which is not factored into the analysis. Each new segment requires additional registers and clocking hardware. Because of this, the optimum number of pipeline segments (S_{opt}) ought to be thought of as a probable upper limit to the number of useful pipeline segments that a particular processor can employ.

2.2.3 Performance

High clock rates with small pipeline segments may or may not produce better performance. Indeed, given problems in wire delay scaling, there is an immediate question of how projected clock rates are to be achieved. There are two basic factors enabling clock rate advances: 1) increased control over clock overhead, and 2) an increased number of segments in the pipelines. Figure 2.4 shows that the length (in gate delays)

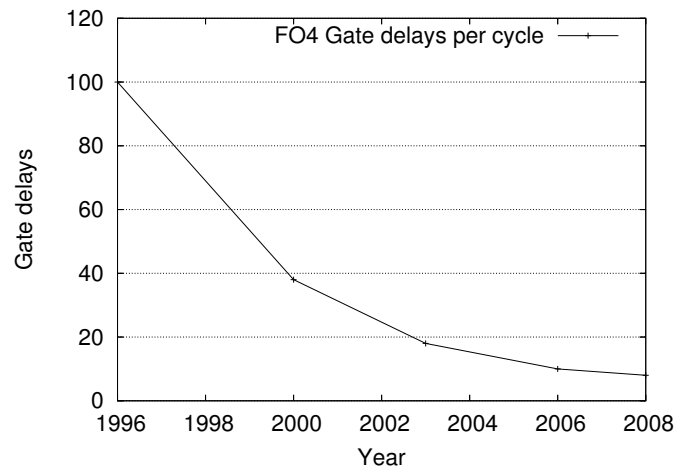


Figure 2.4 Number of FO4 gate delays allowed in a cycle.

of a pipeline segment has decreased significantly, probably by more than five times, measured in units of standard gate (fan-out of four, FO4) delays.

Low clock overhead (small C) may enable increased pipeline segmentation, but performance doesn't correspondingly improve unless we also decrease the probability of pipeline disruption, b . In order to accomplish this high clock rate, processors also employ large branch table buffers and branch vector prediction tables, significantly decreasing delays due to branching. However, disruptions can also come from cache misses and this requires another strategy: multilevel, very large on-die caches. Often these cache structures occupy 80-90% of the die area. The underlying processor is actually less important than the efficiency of the cache-memory system in achieving performance.

2.3 Die Area and Cost

Cycle time, machine organization and memory configuration determine machine *performance*. Determining performance is relatively straightforward when compared with the determination of overall cost.

A good design achieves an optimum cost–performance tradeoff at a particular target performance. This determines the *quality* of a processor design.

In this section we look at the marginal cost to produce a system as determined by the die area component. Of course the system designer must be aware of significant side effects that die area has on the fixed and other variable costs. For example, a significant increase in the complexity of a design may directly affect its serviceability or its documentation costs, or the hardware development effort and time to market. These effects must be kept in mind, even when it is not possible to accurately quantify their extent.

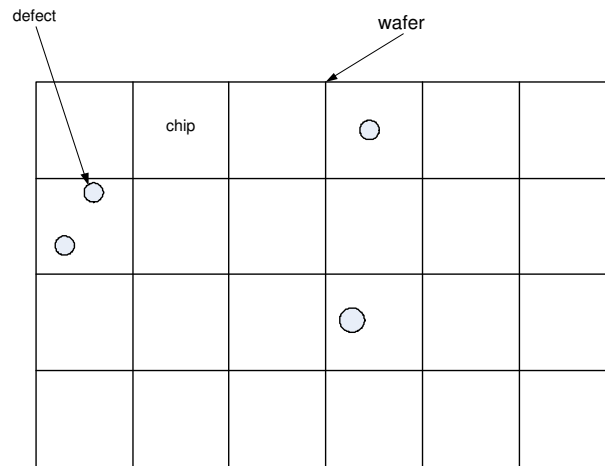


Figure 2.5 Defect distribution on a wafer.

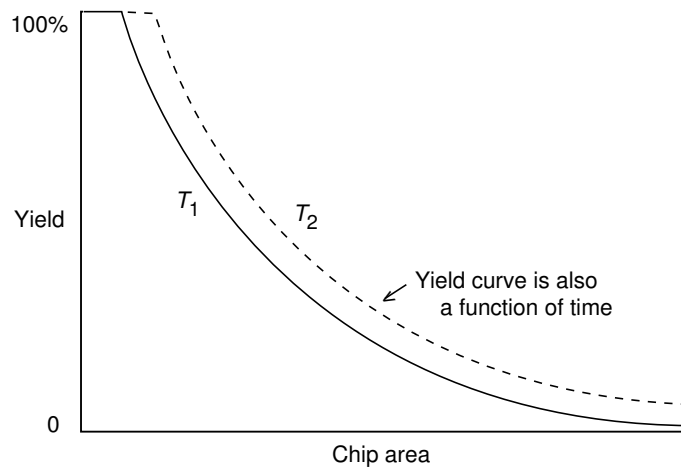


Figure 2.6 Yield versus chip area at various points in time.

Processor Area

SOCs usually use one chip about 10–15 mm on a side. This die is produced in bulk from a larger wafer, perhaps 30 cm in diameter (about 12 inches). It might seem that one could simply expand the chip size and produce fewer chips from the wafer, and these larger chips could realize accommodate any function that the designer might wish to include. But, neither the silicon wafers nor processing technologies are perfect. Defects randomly occur over the wafer surface (Figure 2.5). Large chip areas require an absence of defects over that area. If chips are too large for a particular processing technology, there will be little or no yield. Figure 2.6 illustrates yield versus chip area.

A good design is not necessarily the one that has the maximum yield. Reducing the area of a design below a certain amount has only a marginal effect on yield. Additionally, small designs waste area because there is a required area for pins and for

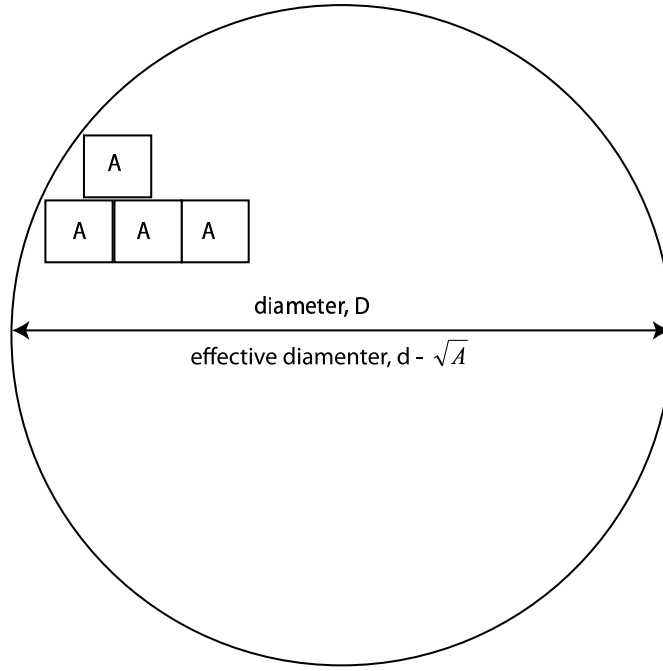


Figure 2.7 Number of die (of area A) on a wafer of diameter d .

separation between adjacent die on a wafer.

The area available to a designer is a function of the manufacturing processing technology. This includes the purity of the silicon crystals, the absence of dust and other impurities, and the overall control of the process technology. Improved manufacturing technology allows larger die to be realized with higher yields. As photolithography and process technology improve their design parameters do not scale uniformly. The successful designer must be aggressive enough in anticipating the movement of technology so that, although early designs may have low yield, with the advance of technology the design life is extended and the yield greatly improves, thus allowing the design team to amortize fixed costs over a broad base of products.

Suppose a die with square aspect ratio has area A . About N of these dice can be realized in a wafer of diameter d (Figure 2.7):

$$N \approx \frac{\pi}{4A} \left(d - \sqrt{A} \right)^2$$

This is the wafer area divided by the die area with diameter correction. Now suppose there are N_G good chips and N_D point defects on the wafer. Even if $N_D > N$, we might expect several good chips, since the defects are randomly distributed and several defects would cluster on defective chips, sparing a few good chips.

Following the analysis of Ghandi [8], suppose we add a random defect to a wafer; N_G/N is the probability that the defect ruins a good die. In other words, the change in the number of good die (N_G), with respect to the change in the number of defects (N_D), is:

$$\begin{aligned}\frac{dN_G}{dN_D} &= -\frac{N_G}{N} \\ \frac{1}{N_G}dN_G &= -\frac{1}{N}dN_D.\end{aligned}$$

Integrating and solving

$$\ln N_G = -\frac{N_D}{N} + C$$

To evaluate C note that when $N_G = N$ then $N_D = 0$; so C must be $\ln N$.

Then the yield is

$$\text{Yield} = \frac{N_G}{N} = e^{-N_D/N}.$$

This describes a Poisson distribution of defects.

If ρ_D is the defect density per unit area, then

$$N_D = \rho_D \times \text{wafer area}$$

For large wafers $d \gg \sqrt{A}$, the diameter of the wafer is significantly larger than the die side and

$$(d - \sqrt{A})^2 \approx d^2$$

and

$$\frac{N_D}{N} = \rho_D A,$$

so that

$$\text{Yield} = e^{-\rho_D A}.$$

Figure 2.8 shows the projected number of good die as a function of die area for several defect densities. Currently a modern fab facility would have ρ_D between 0.15 – 0.5 depending on the maturity of the process and the expense of the facility.

Large die sizes are very costly. Doubling the die area has a significant effect on yield for already large $\rho_D \cdot A$ (i.e., ≈ 5 –10 or more). Thus, the large die designer gambles that technology will lower ρ_D in time to provide a sufficient yield for a profitable product.

Processor Sub-Units

Within a system or processor, the amount of area a particular sub-unit of a design occupies is a primary measure of its cost. In making design choices or in evaluating the

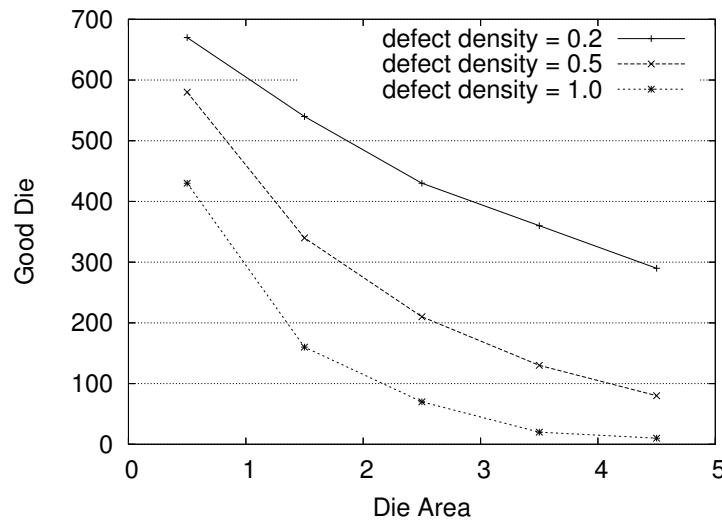


Figure 2.8 Number of good die versus die area for several defect densities.

relative merits of a particular design choice, it is frequently useful to use the principle of *marginal utility*: assume we have a complete base design and some additional pins/area available to enhance the design. We select the design enhancement that best uses the available pins and area. In the absence of pinout information, we assume that area is a dominant factor in a particular tradeoff.

The obvious unit of area is *millimeters*², but since photolithography and geometries' resulting minimum feature sizes are constantly shifting, a dimensionless unit is preferred. Mead and Conway [9] proposed the unit λ , the fundamental resolution, the distance from which a geometric feature on any one layer of mask may be positioned from another. The minimum size for a diffusion region would be 2λ with a necessary allowance of 3λ between adjacent diffusion regions.

If we start with a device $2\lambda \times 2\lambda$ then a device of nominal $2\lambda \times 2\lambda$ can extend to $4\lambda \times 4\lambda$. We need at least 1λ isolation from any other device or $25\lambda^2$ for the overall device area. Thus, a single transistor is $4\lambda^2$, positioned in a minimum region of $25\lambda^2$.

The minimum feature size (f) is the length of one polysilicon gate, or the length of one transistor, $f = 2\lambda$. Clearly, we could define our design in terms of λ^2 and any other processor feature (gate, register size, etc.) can be expressed as a number of transistors. Thus, the selection of the area unit is somewhat arbitrary. However, a better unit represents primary architectural tradeoffs. One useful unit is the register bit equivalent (**rbe**). This is defined to be a six-transistor register cell and represents about $2700\lambda^2$. This is significantly more than six times the area of a single transistor, since it includes larger transistors, their interconnections, and necessary inter-bit isolating spaces.

The unit **rbe** is defined as the area of a six-transistor static cell with high bandwidth—a register bit isolated from its input/output circuitry. A SRAM static cell with lower bandwidth would use less area and a DRAM-bit cell would use still less. Empirically, they would have the relationship shown in Table 2.2.

In the table the area for the register file is determined by the number of register bits

Item Size in <i>rbe</i>	
1 register bit (rbe)	1.0 rbe
1 static RAM bit in an on-chip cache	0.6 rbe
1 DRAM bit	0.1 rbe
rbe corresponds to (in feature size: <i>f</i>)	1 rbe = $675f^2$
Item	Size in <i>A</i> Units
<i>A</i> corresponds to 1 mm ² with <i>f</i> = 1μ.	
1 <i>A</i> or about	= $f^2 \times 10^6$ (<i>f</i> in microns) ≈ 1481 rbe
A simple integer file (1 read + 1 read/write) with 32 words of 32 bits/word or about	= 1444 rbe ≈ 1 <i>A</i> (= 0.975 <i>A</i>)
A 4KB direct mapped cache or about	= 23,542 rbe ≈ 16 <i>A</i>
Generally a simple cache (whose tag and control bits are less than 1/5th the data bits) uses	= 4 <i>A</i> / <i>KB</i>
Simple processors (approximation)	
A 32-bit processor (no cache and no floating point)	= 50 <i>A</i>
A 32-bit processor (no cache but includes 64-bit floating point)	= 100 <i>A</i>
A 32b (signal)processor, as above, with vector facilities but no cache or vector memory	= 200 <i>A</i>
Area for inter-unit latches, buses, control and clocking	allow an additional 50% of the processor area.
Xilinx FPGA	
A slice (2 LUTs + 2 FFs + MUX)	= 700 rbe
A configurable logic block (4 slices) Virtex 4	= 2,800 rbe ≈ 1.9 <i>A</i>
A 18Kb BlockRAM	= 12,600 rbe ≈ 8.7 <i>A</i>
An Embedded PPC405 core	≈ 250 <i>A</i>

Table 2.2 Summary of relative areas.

Feature size (in micron)	Number of A per mm^2
1.000	1.00
0.350	8.16
0.130	59.17
0.090	123.46
0.065	236.69
0.045	493.93

Table 2.3 Density in A units for various feature sizes. One A is 1481 **rbe**.

and the number of ports (P) available to access the file:

$$\text{Area of register file} = (\text{number of regs} + 3P) (\text{bits per reg} + 3P) \text{ rbe.}$$

The cache area uses the SRAM bit model and is determined by the total number of cache bits, including the array, directory and control bits.

The number of **rbes** on a die or die section rapidly becomes very large, so it is frequently easier to use a still larger unit. We refer to this unit simply as A , and define it as 1 mm^2 of die area at $f = 1\mu$. This is also the area occupied by a 32 by 32 bit 3-ported register file or 1481 **rbe**.

Transistor density, **rbe** and A all scale as the square of the feature size. As seen from Table 2.3, for feature size f , the number of A in 1 mm^2 is simply $1/f^2$. There are almost 500 times as many transistors of **rbe** in 1 mm^2 of the latest feature size (45 nm) as there are with the reference 1 micron feature size.

2.4 Ideal and Practical Scaling

As feature sizes shrink and transistors get smaller, one expects the transistor density to improve with the square of the change in feature size. Similarly transistor delay (or gate delay) should decrease linearly with feature size (corresponding to the decrease in capacitance). Practical scaling is different as wire delay and wire density does not scale as expected. Wire delay remains almost constant as feature sizes shrink since the increase in resistance offsets the decrease in length and capacitance. Figure 2.10 illustrates the increasing dominance of wire delay over gate delay especially in feature sizes less than 0.13μ . Similarly for feature sizes below 0.20μ transistor density improves at somewhat less than the square of the feature size. A suggested scaling factor of 1.5 is commonly considered more accurate, as shown in Figure 2.9. That is scaling occurs at $(f_1/f_2)^{1.5}$ rather than at $(f_1/f_2)^2$. What actually happens during scaling is more complex. Not only does the feature size shrink but other aspects of a technology also change and usually improve. Thus copper wires become available as well as many more wiring layers and improved circuit designs. So that the effects of wire limitations are largely mitigated, so long as the designer is able to use all the attributes of the new technology generation. So the simple scaling of a design (without altering the design) might only scale as 1.5 but a new implementation taking advantage of all technology

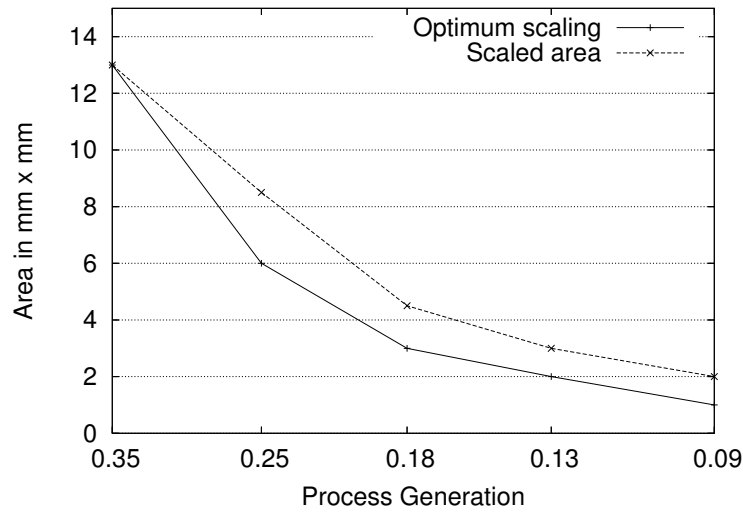


Figure 2.9 Area scaling with optimum and “practical” shrinkage.

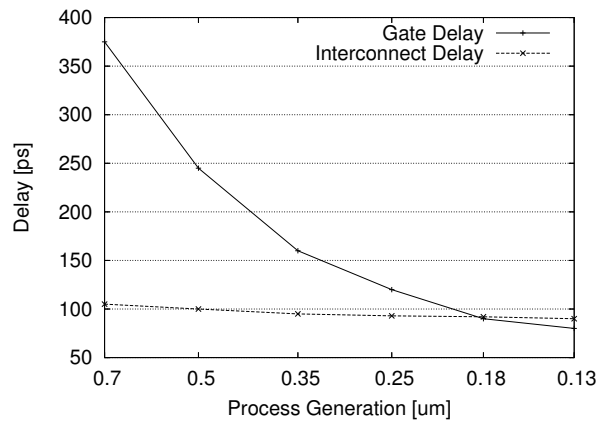


Figure 2.10 The dominance of wire delay over gate delay.

features could scale at 2. For simplicity in the remainder of the text we’ll use ideal scaling with the understanding as above.

Study 2.1 A Baseline SOC Area Model

The key to efficient system design is *chip floor planning*. The process of chip floor planning is not much different from the process of floor-planning a residence. Each functional area of the processor must be allocated sufficient room for its implementation. Functional units that frequently communicate must be placed close together. Sufficient room must be allocated for connection paths.

To illustrate possible tradeoffs that can be made in optimizing the chip floor plan,

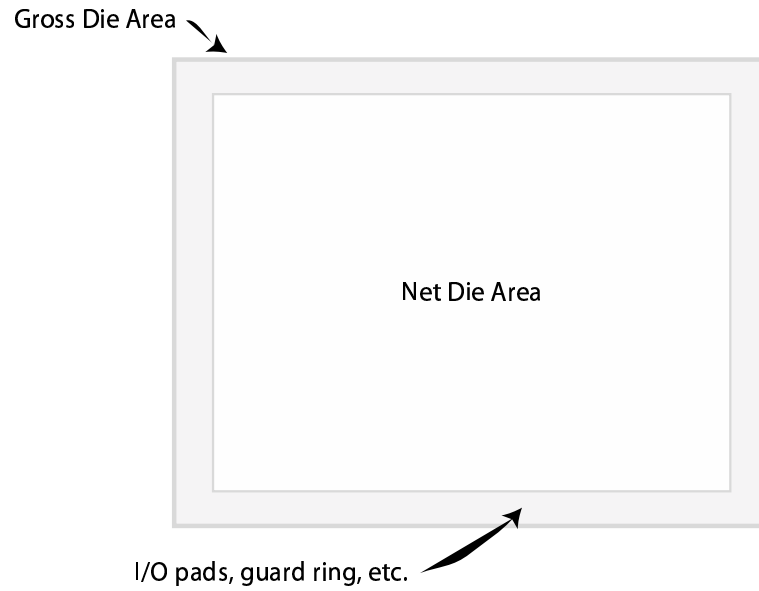


Figure 2.11 Net die area.

we introduce a baseline system with designated areas for various functions. The area model is based upon empirical observations made of existing chips, design experience and, in some cases, logical deduction (e.g. the relationship between a floating-point adder and an integer ALU). The chip described here ought not to be considered optimal in any particular sense, but rather a typical example of a number of designs in the marketplace today.

The Starting Point The design process begins with an understanding of the parameters of the semiconductor process. Suppose we expect to be able to use a manufacturing process that has a defect density of 0.2 defect per square centimeter; for economic reasons, we target an initial yield of about 80%.

$$Y = e^{-\rho_D A},$$

where $\rho_D = 0.2$ defect/cm², $Y = 0.8$. Then

$$A \doteq 110 \text{ mm}^2$$

or approximately 1 cm².

So the chip area available to us is 110 mm². This is the total die area of the chip, but such things as pads for the wire bonds that connect the chip to the external world, drivers for these connections, and power supply lines all act to decrease the amount of chip area available to the designer. Suppose we allow 20% of the chip area—usually around the periphery of the chip—to accommodate these functions, then the net area will be 88 mm².

Feature Size The smaller the feature size, the more logic that can be accommodated within a fixed area. At $f = 0.13\mu$ we have about 5,200A or area units in 88 mm^2 .

The Architecture Almost by definition each system is different with different objective. For our example assume that we need the following:

- A small 32b core processor with an 8KB I cache and a 16KB D cache
- Two 32b vector processors, each with 16 banks of $1K \times 32b$ vector memory; an 8KB I cache and a 16KB D cache for scalar data.
- A bus control unit.
- Directly addressed application memory of 128KB.
- A shared L2 cache.

An Area Model The following is a breakdown of the area required for various units used in the system.

<i>Unit</i>	<i>Area</i>
Core Processor (32^b)	100A
Core cache (24KB)	96A
Vector Processor #1	200A
Vector Registers & cache #1	256 + 96A
Vector Processor #2	200A
Vector Registers & cache #2	352A
Bus and bus control (50%)	see below 650A
Application memory (128KB)	512A
Subtotal	2,462A

Latches, Buses, and (Interunit) Control For each of the functional units, there is a certain amount of overhead to accommodate nonspecific storage (latches), interunit communications (buses), and interunit control. This is allocated as 10% overhead for latches and 40% overhead for buses, clocking, and overall control.

Total System Area The designated processor elements and storage occupy 2,462A. This leaves a net of $5200 - 2462 = 2738A$ available for cache. Note that the die is highly storage oriented. The remaining area will be dedicated to the L2 cache.

Cache Area The net area available for cache is 2,738A. However, bits and pieces that may be unoccupied on the chip are not always useful to the cache designer. These pieces must be collected into a reasonably compact area that accommodates efficient cache designs.

For example, where the available area has a large height-to-width (aspect) ratio, it may be significantly less useful than a more compact or square area. In general, at this

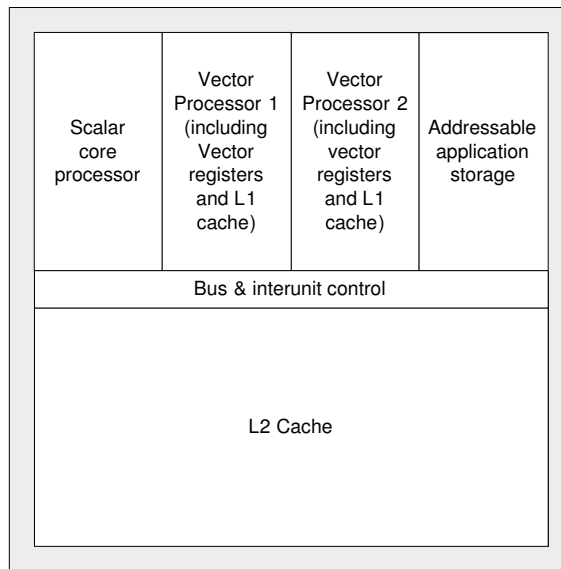


Figure 2.12 A baseline die floorplan.

early stage of microprocessor floor planning, we allocate another 10% overhead to aspect ratio mismatch. This leaves a net available area for cache of about 2,464A.

This gives us about 512KB for the L2 cache. Is this reasonable? At this point all we can say is that this much cache fits on the die. We now must look to the application and determine if this allocation gives the best performance. Perhaps a larger application storage or another vector processor and a smaller L2 would give better performance. Later in the text we consider such performance issues.

An example baseline floorplan is shown in Figure 2.12. Summary of Area Design Rules:

1. Compute target chip size from target yield and defect density.
2. Compute die cost and determine whether satisfactory.
3. Compute net available area. Allow 20% (or other appropriate factor) for pins, guard ring, power supplies, etc.
4. Determine the **rbe** size from the minimum feature size.
5. Allocate area based on a trial system architecture until the basic system size is determined.
6. Subtract basic system size (5) from net available area (3). This is the die area available for cache and storage optimization.

Note that in this study (as in most implementations) most of the die area is dedicated to storage of one type or another. The basic processor(s) area is around 20%, allowing

Type	Power/die	Source & Environment
Cooled high power	70 watts	Plug-in; Chilled
High Power	10-50 watts	Plug -in; fan
Low power	0.1-2 watts	Rechargeable battery
Very low power	1-100 mwatts	AA batteries
Extremely low power	1-100 μ watts	Button battery

Table 2.4 Some power operating environments.

for a partial allocation of bus and control area. Thus, however rough our estimate of processor core and vector processor area, it is likely to have little effect on the accuracy of the die allocation so long as our storage estimates are accurate.

2.5 Power

Growing demands for wireless and portable electronic appliances have focused much attention recently on power consumption. The SIA roadmap points to increasingly higher power for microprocessor chips because of their higher operating frequency, higher overall capacitance, and larger size. Some power environments are shown in Table 2.4.

At the device level, total power dissipation (P_{total}) has two major sources: dynamic or switching power and static power caused by leakage current:

$$P_{total} = \frac{CV^2 freq}{2} + I_{leakage}V$$

where C is the device capacitance, V is the supply voltage, $freq$ is the device switching frequency, $I_{leakage}$ is the leakage current. Until recently switching loss was the dominant factor in dissipation, but now static power is increasing. On the other hand, gate delays are roughly proportional to $CV / (V - V_{th})^2$, where V_{th} is the threshold voltage (for logic level switching) of the transistors.

As feature sizes decrease, so do device sizes. Smaller device sizes result in reduced capacitance. Decreasing the capacitance decreases both the dynamic power consumption and the gate delays. As device sizes decrease, the electric field applied to them becomes destructively large. To increase the device reliability, we need to reduce the supply voltage V . Reducing V effectively reduces the dynamic power consumption, but results in an increase in the gate delays. We can avoid this loss by reducing V_{th} . On the other hand, reducing V_{th} increases the leakage current, and therefore, the static power consumption. This has an important effect on design and production; there are two device designs that must be accommodated in production:

1. The high speed device with low V_{th} and high static power.
2. The slower device maintaining V_{th} and V at the expense of circuit density and low static power.

Type	Energy Capacity	Duty cycle/lifetime	at Power
Rechargeable	10,000 mAh	50 Hours (10-20% duty)	400 mw-4 w
2 x AA	4,000 mAh	1/2 year (10-20% duty)	1-10 mw
Button	40 mAh	5 years (always on)	1 μ w

Table 2.5 Battery capacity and duty cycle.

In either case we can reduce switching loss by lowering the supply voltage, V . Chen et al. showed that the drain current is proportional to

$$I = (V - V_{th})^{1.25}$$

where again V is the supply voltage.

From our discussion above, we can see that the signal transition time and frequency scale with the charging current. So the maximum operating frequency is also proportional to $\frac{(V - V_{th})^{1.25}}{V}$. For values of V and V_{th} of interest this means that frequency scales with the supply voltage, V .

Assume V_{th} is $0.6V$, suppose we reduce the supply voltage by $1/2$, say from 3V to 1.5V, the operating frequency is also reduced by about $1/2$. So reducing the supply voltage by half also reduces the operating frequency by half.

Now by the power equation (since the voltage and frequency were halved) the total power consumption is $1/8$ of the original. Thus, if we take an existing design optimized for frequency and modify that design to operate at a lower voltage, the frequency is reduced by approximately the cube root of the original (dynamic) power:

$$\frac{freq_1}{freq_2} = \sqrt[3]{\frac{P_2}{P_1}}$$

It is important to understand the distinction between scaling the frequency of an existing design and that of a power-optimized implementation. Power-optimized implementations differ from performance-optimized implementations in several ways. Power-optimized implementations use less chip area not only because of reduced requirements for power supply and clock distributions, but also, and more importantly, because of reduced performance targets. Performance oriented designs use a great deal of area to achieve marginally improved performance- very large floating point units, minimum-skew clock distribution networks, or maximally sized caches. Power dissipation, not performance, is the most critical issue for applications such as portable and wireless processors running on batteries. Some battery capacities are shown in table 2.5.

For SOC designs to run on battery power for an extended period, the entire system power consumption must remain very small (on the order of a milliwatt).

As a result, power management must be implemented from the system architecture and operating system down to the logic gate level.

2.6 Area- time- power tradeoffs in processor design

Processor design tradeoffs are quite different for our two general classes of processors:

1. Workstation type processor. These designs are oriented to high clock frequency, and AC power sources (excluding laptops). Since they are not area limited as the cache occupies most die area, the designs are highly elaborated (superscalar with multi threading).
2. System on a chip (SOC). Processors here are generally simpler in control structure but may be quite elaborate in execution facilities (e.g. DSP). Area is a factor as is design time and power.

2.6.1 Workstation type processor

To achieve general purpose performance, the designer assumes ample power. The most basic tradeoff is between high clock rates and the resulting power consumption. Up until the early '90s the bipolar technology of ECL (emitter coupled logic) was dominant in high performance applications (mainframes and supercomputers). At power densities of 80 watts/cm² the module package required some form of liquid cooling. An example from this period is the Hitachi M-880 (Figure 2.13). A 10x10 cm module consumed 800 watts. The module contained 40 die, sealed in helium gas with chilled water pumped across a water jacket at the top of the module. As CMOS performance approached bipolar's, the extraordinary cost of such a cooling system could no longer be sustained, and the bipolar era ended—see Figure 2.14. Now CMOS approaches the same power densities, and similar cooling techniques may be reconsidered.

2.6.2 System on a chip

System on a chip type implementations have a number of advantages. The requirements are generally known. So memory sizes and real time delay constraints can be anticipated. Processors can be specialized to a particular function. In doing so usually clock frequency (and power) can be reduced as performance can be regained by straightforward concurrency in the architecture (e.g. use of a simple VLIW for DSP applications). The disadvantages of SOC compared to processor chips are available design time/effort and intra die communications between functional units. In SOC the market for any specific system is relatively small hence the extensive custom optimization used in processor dies is difficult to sustain, so off-the-shelf core processor designs are commonly used. As the storage size for programs and data may be known at design time, specific storage structures can be included on chip. These are either SRAM or a specially designed DRAM (as ordinary DRAM uses an incompatible process technology). With multiple storage units, multiple processors (some specialized, some generic) and specialized controllers, the problem is designing a robust bus hierarchy to ensure timely communications. A comparison between the two design classes is shown in Table 2.6.

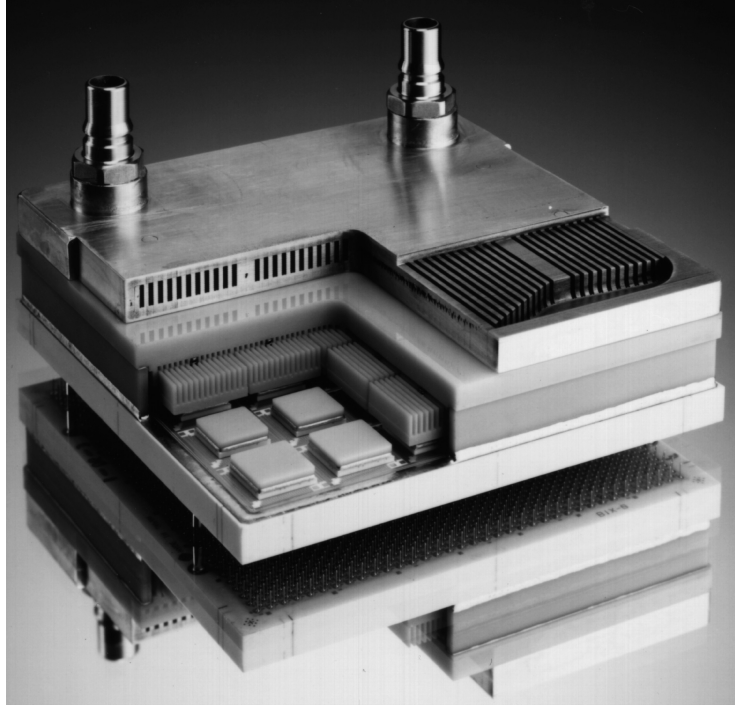


Figure 2.13 Hitachi processor module. The Hitachi M-880 was introduced about 1991 [7]. Module is 10.6×10.6 cm, water cooled and dissipated 800 watts.

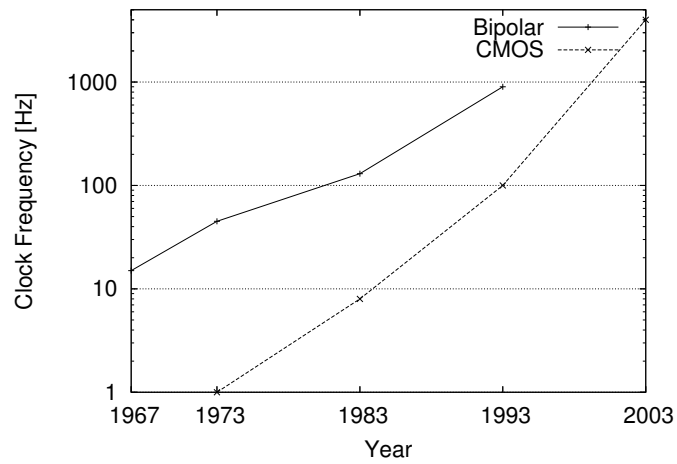


Figure 2.14 Processor frequency for bipolar and CMOS over time.

	Processor on a chip	SOC
Area used by storage	80% cache	50% ROM/RAM
Clock frequency	4 GHz	1/2 GHz
Power	≥ 50 Watts	≤ 10 Watts
Memory	≥ 1 GB DRAM	mostly on die

Table 2.6 A typical processor die compared with a typical SOC die.

2.7 Reliability

The fourth important design dimension is reliability; also referred to as dependability and fault-tolerance. As with cost and power there are many more factors which contribute to reliability than what is done on a processor or SOC die.

Reliability is related to die area, clock frequency and power. Die area increases the amount of circuitry and the probability of a fault but it also allows the use of error correction and detection techniques. Higher clock frequencies increase electrical noise and noise sensitivity. Faster circuits are smaller and more susceptible to radiation.

Not all failures or errors produce faults and indeed not all faults result in incorrect program execution. Faults, if detected, can be masked by error correcting codes, instruction retry or functional reconfiguration.

First some definitions:

1. A *failure* is a deviation from a design specification.
2. An *error* is a failure that results in an incorrect signal value
3. A *fault* is an error that manifests itself as an incorrect logical result.
4. A *physical fault* is a failure caused by environment such as aging, radiation, temperature or temperature cycling, etc. The probability of physical faults increase with time.
5. A **design fault** is a failure caused by a design implementation that is inconsistent with the design specification. Usually design faults occur early in the lifetime of a design and are reduced or eliminated over time.

Dealing with Physical Faults

From a system point of view, we need to create processor and sub-unit configurations that are robust over time.

Let the probability of a fault occurrence be $P(t)$, and let T be the *mean time between faults* or the MTBF. So if λ is the fault rate, then:

$$\lambda = \frac{1}{T}$$

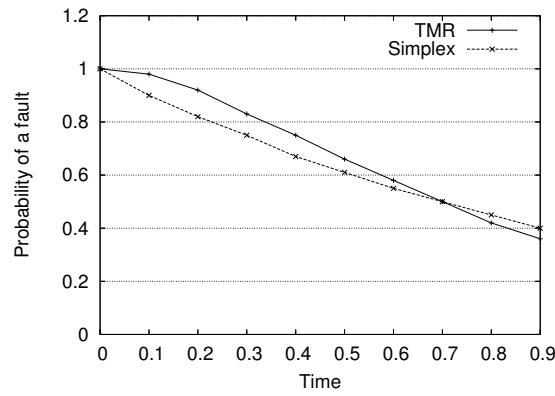


Figure 2.15 TMR reliability compared to simplex reliability.

Now imagine that faults occur on the time axis in particular time units, separated with mean, T . Using the same reasoning that we used to develop the Poisson yield equation we can get the Poisson fault equation.

$$P(t) = e^{-\frac{t}{T}} = e^{-t\lambda}$$

Redundancy is an obvious approach to improved reliability (lower $P(t)$). A well known technique is *triple modular redundancy* or TMR. Three processors execute the same computation and compare results. A voting mechanism selects the output on which at least two processors agree. TMR works but only up to a point. Beyond the obvious problem of the reliability of the voting mechanism there is a problem with the sheer amount of hardware. Clearly as time, t , approaches T we expect to have more faults in the TMR system than in a simple simplex system. Indeed, the probability of a TMR fault (any two out of three processor faults) exceeds the simplex system when:

$$t = T \times \log_e 2$$

Most fault-tolerant design involves simpler hardware built around:

- Error detection. The use of parity, residue and other codes are essential to reliable system configurations.
- Instruction (action) retry. Once a fault is detected, the action can be retried to overcome transient errors.
- Error correction. Since most of the system is storage and memory an error correction code (ECC) can be effective in overcoming storage faults.
- Reconfiguration. Once a fault is detected it may be possible to reconfigure parts of the system so that the failing sub-system is isolated from further computation.

Note that with error detection efficient, reliable system configurations are limited. As a minimum most systems should incorporate error detection on all parts of essential system components and selectively use ECC and other techniques to improve reliability.